



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 04

NOMBRE COMPLETO: Del Toro Ortiz Juan Pablo Yasbeth

N° de Cuenta: 317031814

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 7

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 07/03/2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

Desarrollo de un modelo tridimensional de una grúa articulada mediante la conexión de piezas independientes que heredan el movimiento de la base principal.

Componentes del modelo:

- **Cuerpo:** Una cabina amarilla apoyada sobre una base de pirámide cuadrangular gris.
- **Brazo de 3 partes:** Una estructura de tres segmentos largos que se extienden desde la cabina.
- **4 articulaciones:** Esferas que sirven como puntos de unión y permiten la rotación de cada sección.
- **1 canasta:** Un cubo gris al final del recorrido para transportar al operador.
- **4 llantas:** Cilindros negros en la base que rotan de manera independiente.

Controles: Se utilizan las teclas de la **F** a la **K** para accionar el movimiento de las articulaciones y el giro de las llantas.

Cambio en el código del cilindro (número para efecto más circular)

```
287 CrearCubo();//índice 0 en MeshList
288 CrearPiramideTriangular();//índice 1 en MeshList
289 CrearCilindro(15, 1.0f);//índice 2 en MeshList //cambio a 15 para llantas
290 CrearCono(25, 2.0f);//índice 3 en MeshList
291 CrearPiramideCuadrangular();//índice 4 en MeshList
292 CreateShaders();
293
```

Sección de la cabina:

```
348 // Reiniciar la matriz de modelo con valor de la matriz identidad
349 model = glm::mat4(1.0);
350
351 // =====
352 // CABINA
353 // =====
354 // Traslación inicial para posicionar toda la grúa en el mundo
355 model = glm::translate(model, glm::vec3(0.0f, 2.0f, -4.0f));
356 // Guardamos esta transformación (posición global) para que los hijos la hereden
357 // pero sin la escala que aplicaremos a continuación.
358 modelaux = model;
359
360 // Transformaciones propias de la Cabina
361 model = glm::scale(model, glm::vec3(4.0f, 2.0f, 3.0f));
362 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
363 color = glm::vec3(1.0f, 1.0f, 0.0f); // Amarillo
364 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
365 meshList[0]->RenderMesh(); // Dibuja el cubo de la cabina
366
```

Sección de los brazos:

```
367 // =====
368 // BRAZO 1
369 // =====
370 // Recuperamos la matriz de la cabina (solo posición, sin escala)
371 model = modelaux;
372
373 // articulación 1: Posicionamos el brazo respecto a la cabina (ej. en el techo)
374 model = glm::translate(model, glm::vec3(-0.5f, 0.2f, 0.0f));
375 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
376
377 // primer brazo que conecta con la cabina
378 // Aplicamos el desplazamiento para que rote desde un extremo
379 model = glm::translate(model, glm::vec3(-1.0f, 2.0f, 0.0f));
380 model = glm::rotate(model, glm::radians(135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
381 modelaux = model; // Guardamos para la articulación 2
382
383 model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
384 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
385 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
386 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
387 color = glm::vec3(1.0f, 1.0f, 0.0f);
388 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
389 meshList[0]->RenderMesh();
390
391 // =====
392 // BRAZO 2
393 // =====
394 // Para descartar la escala cargamos la matrix auxiliar
395 model = modelaux;
396
397 // articulación 2
398 model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
399 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
400 modelaux = model;
401
402 // dibujar una pequeña esfera
403 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
404 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
405 sp.render();
406
407 model = modelaux;
408 // segundo brazo
409 model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f));
410 modelaux = model;
411
412 model = glm::scale(model, glm::vec3(1.0f, 5.0f, 1.0f));
413 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
414 color = glm::vec3(1.0f, 1.0f, 0.0f);
415 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
416 meshList[0]->RenderMesh();
417
```

```

418 // =====
419 // BRAZO 3
420 // =====
421 // Para descartar la escala cargamos la matrix auxiliar del brazo 2
422 model = modelaux;
423
424 // articulación 3 (al final del brazo 2)
425 model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f));
426 // Rotación de la articulación (controlada por teclado) + los 135 grados de la imagen
427 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3() + 35.0f), glm::vec3(0.0f, 0.0f, 1.0f));
428 modelaux = model;
429
430 // dibujar una pequeña esfera
431 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
432 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
433 sp.render();
434
435 model = modelaux;
436 // tercer brazo
437 // Se traslada en X porque, tras la rotación, el brazo se extiende "hacia adelante"
438 model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
439 modelaux = model;
440
441 model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
442 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
443 color = glm::vec3(1.0f, 1.0f, 0.0f);
444 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
445 meshList[0] -> RenderMesh();
446
447 // Para descartar la escala cargamos la matrix auxiliar del brazo 3
448 model = modelaux;
449
450 // articulación 4 (punto final del brazo 3)
451 // Nos movemos al extremo del brazo 3 (que mide 5 unidades de largo)
452 model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
453 // Rotación de la canasta (puedes usar una variable nueva o dejarla fija)
454 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
455 modelaux = model;
456
457 // dibujar una pequeña esfera (pivote de la canasta)
458 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
459 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
460 sp.render();
461

```

Sección de la canasta:

```

462
463 // =====
464 // CANASTA
465 // =====
466 model = modelaux;
467 // La desplazamos un poco hacia abajo y hacia adelante para que cuelgue del pivote
468 model = glm::translate(model, glm::vec3(0.5f, -0.7f, 0.0f));
469 modelaux = model;
470
471 // Escalamos para que sea un cubo donde quepa una persona (
472 model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.7f));
473 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
474 color = glm::vec3(0.7f, 0.7f, 0.7f); // Gris claro / Metálico
475 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
476 meshList[0] -> RenderMesh(); // Dibuja el cubo
477

```

Base de la cabina:

```
479 // =====
480 // BASE DE LA CABINA (Pirámide Cuadrangular)
481 // =====
482
483 // Reiniciamos a la matriz de la cabina (posición global)
484 // para no heredar rotaciones ni escalas de los brazos o la canasta.
485 model = glm::mat4(1.0);
486 model = glm::translate(model, glm::vec3(0.0f, 2.0f, -4.0f)); // Misma posición base que la cabina
487
488 // La movemos hacia abajo para que quede debajo del cubo amarillo
489 model = glm::translate(model, glm::vec3(0.0f, -0.7f, 0.0f));
490
491 // Guardamos para las llantas
492 modelaux = model;
493
494 // Escalamos la pirámide para que sea una base sólida
495 model = glm::scale(model, glm::vec3(4.0f, 2.0f, 3.3f));
496
497 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
498 color = glm::vec3(0.5f, 0.5f, 0.5f); // Gris oscuro para la base
499 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
500
501 // Dibujamos la pirámide cuadrangular (índice 4 según tu código inicial)
502 meshList[4]->RenderMesh();
503
```

```
504 // =====
505 // LLANTAS (Relación con la Base/Pirámide)
506 // =====
507
508 // --- LLANTA 1 ---
509 model = modelaux;
510 // X: 2.2f para que sobresalga del ancho (4.0/2 = 2.0)
511 // Y: -0.5f para que esté por debajo del centro de la base
512 // Z: 1.5f para que esté en el frente
513 model = glm::translate(model, glm::vec3(1.5f, -0.8f, 1.8f));
514
515 // Rotaciones para orientar el cilindro correctamente
516 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
517 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
518
519 // Rotación independiente de rodado
520 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 1.0f, 0.0f));
521
522 model = glm::scale(model, glm::vec3(1.0f, 0.6f, 1.0f));
523 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
524 color = glm::vec3(0.1f, 0.1f, 0.1f);
525 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
526 meshList[2]->RenderMeshGeometry(); // Cilindro
527
```

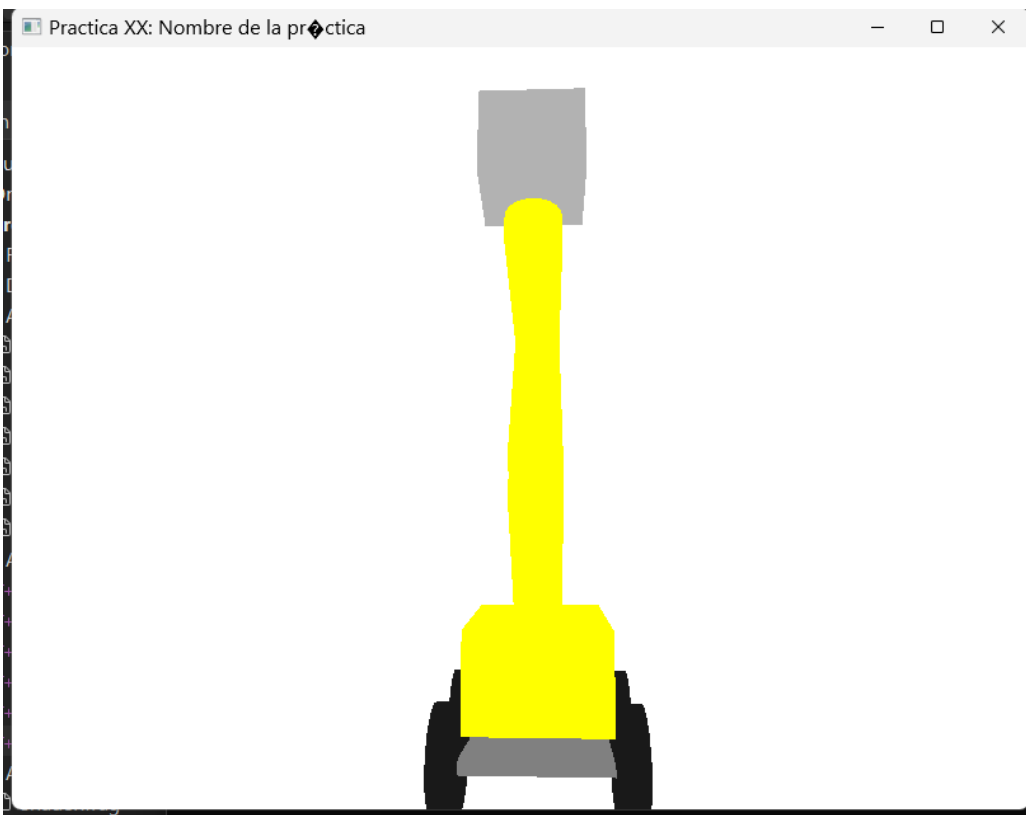
```
528 // --- LLANTA 2 ---
529 model = modelaux;
530 // Cambiamos 1.5f a -1.5f para moverla atras
531 model = glm::translate(model, glm::vec3(-1.5f, -0.8f, 1.8f));
532
533 // Mantiene exactamente las mismas rotaciones para que herede la orientación
534 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
535 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
536
537 // Rotación independiente de rodado
538 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 1.0f, 0.0f));
539
540 model = glm::scale(model, glm::vec3(1.0f, 0.6f, 1.0f));
541 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
542 color = glm::vec3(0.1f, 0.1f, 0.1f);
543 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
544 meshList[2]->RenderMeshGeometry(); // Cilindro
545
```

```

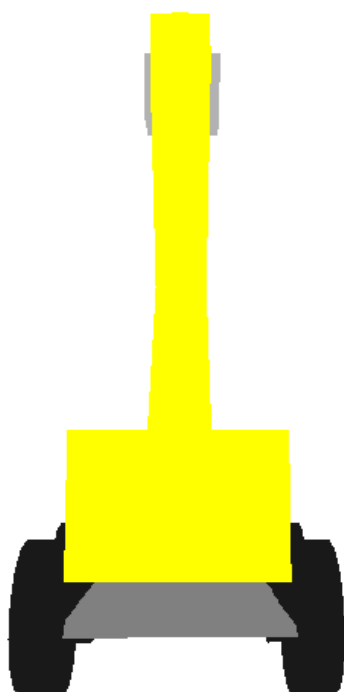
546 // --- LLANTA 3 ---
547 model = modelaux;
548 // Cambiamos 1.8f a -1.8f para moverla al otro lado
549 model = glm::translate(model, glm::vec3(1.5f, -0.8f, -1.8f));
550
551 // Mantiene exactamente las mismas rotaciones para que herede la orientación
552 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
553 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
554
555 // Rotación independiente de rodado
556 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 1.0f, 0.0f));
557
558 model = glm::scale(model, glm::vec3(1.0f, 0.6f, 1.0f));
559 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
560 color = glm::vec3(0.1f, 0.1f, 0.1f);
561 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
562 meshList[2]->RenderMeshGeometry(); // Cilindro
563
564 // -- LLANTA 4 --
565 model = modelaux;
566 // Cambiamos 1.8f y 1.5 a -1.8f y -1.5 para moverla al otro lado atras
567 model = glm::translate(model, glm::vec3(-1.5f, -0.8f, -1.8f));
568
569 // Mantiene exactamente las mismas rotaciones para que herede la orientación
570 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
571 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
572
573 // Rotación independiente de rodado
574 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 1.0f, 0.0f));
575
576 model = glm::scale(model, glm::vec3(1.0f, 0.6f, 1.0f));
577 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
578 color = glm::vec3(0.1f, 0.1f, 0.1f);
579 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
580 meshList[2]->RenderMeshGeometry(); // Cilindro

```

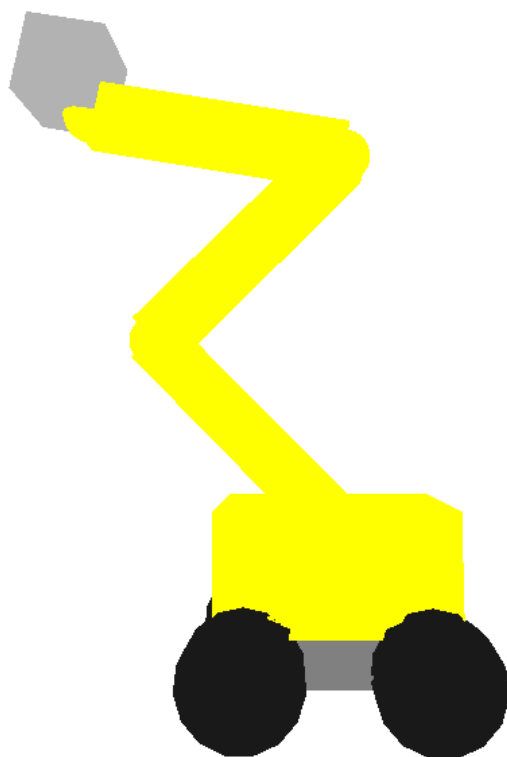
Capturas de pantalla de la ejecución:



Vista frontal

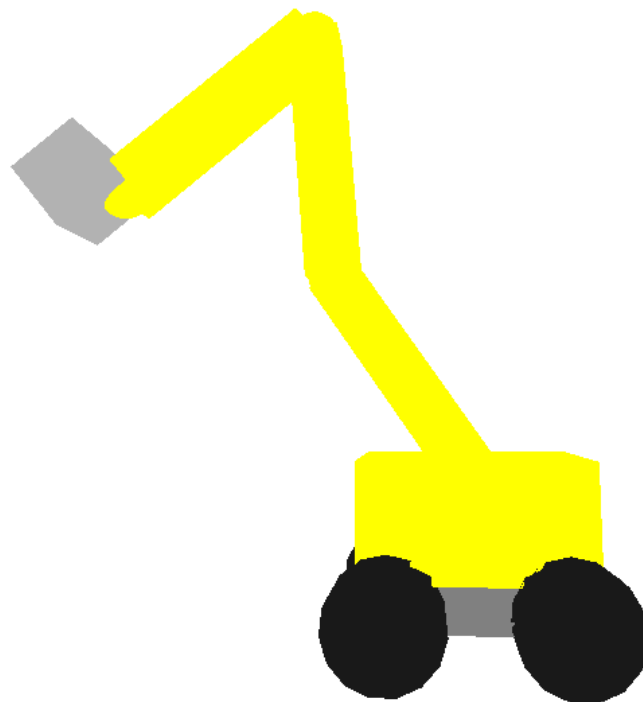
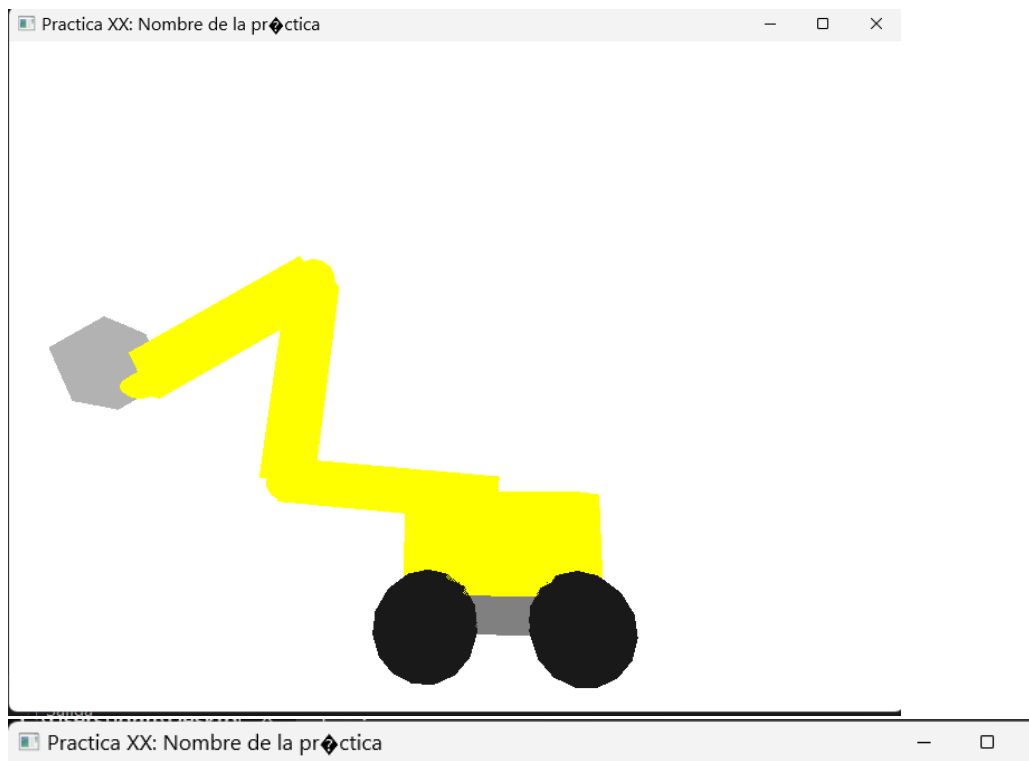


Vista posterior

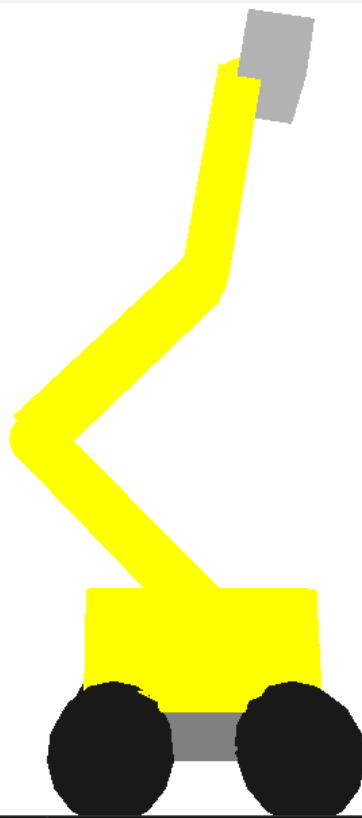


Vista lateral

Movimiento de las piezas:

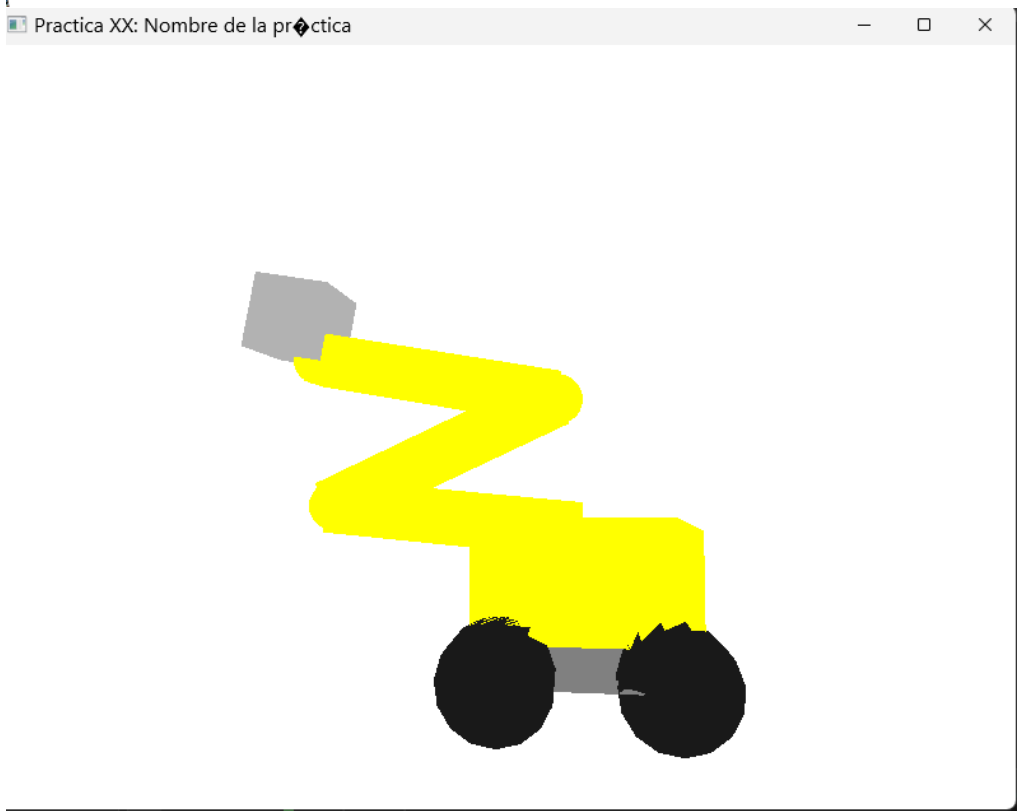
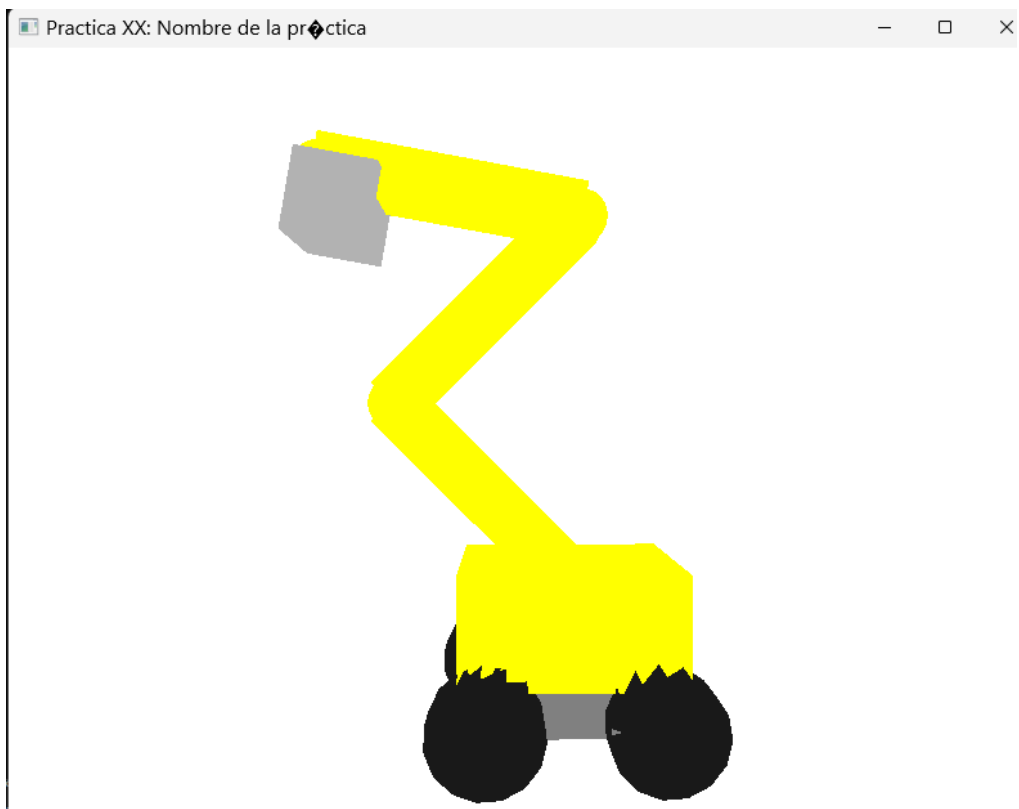


Practica XX: Nombre de la práctica



Practica XX: Nombre de la práctica





2. Problemas presentados. Listar si surgieron problemas a la hora de ejecutar el código

El principal problema al que me enfrente fue crear inicialmente la cabina para ajustar la prioridad e índice de los modelos, además para la creación de las llantas necesite experimentar un poco con las rotaciones y me di cuenta que necesitaba 2 rotaciones para que estas quedaran en la posición deseada

3. Conclusión:

- a. Los ejercicios de la clase: Complejidad, explicación

En esta ocasión no me fue complejo realizar el ejercicio, fuera de hacer la instancia de la cabina como primer elemento, luego de eso el ir agregando elementos resulto hasta cierto punto más sencillo, pero requería cuidado con las rotaciones por los giros sobre el eje de referencia

- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias.

En general creo que en esta ocasión la explicación teórica fue muy extensa y un poco abrumadora, honestamente me costo seguir el paso de la sesión y en mi casa requerí de repasar y estar un rato volviendo a leer el código y entendiendo todo lo realizado

Aparte del punto anterior me resulto muy interesante aprender como se pueden heredar elementos como el escalado, rotación y traslación para los diferentes elementos que se instancian en el modelado y me pareció interesante el tener que ir cuidando las dimensiones de la figura para mantener cierta proporcionalidad con sus elementos.